

ONLINE SESSION · 2.5 HOURS

Session 18

CQRS Pattern

Command Query Responsibility Segregation · Read Models
Spring Data Projections · Domain Events



Dr. ElSayed Mohamed Elsayed Baladoh

Tech Lead · Ph.D. · sayedbaladoh@yahoo.com



Monday · 3:00–5:30 PM

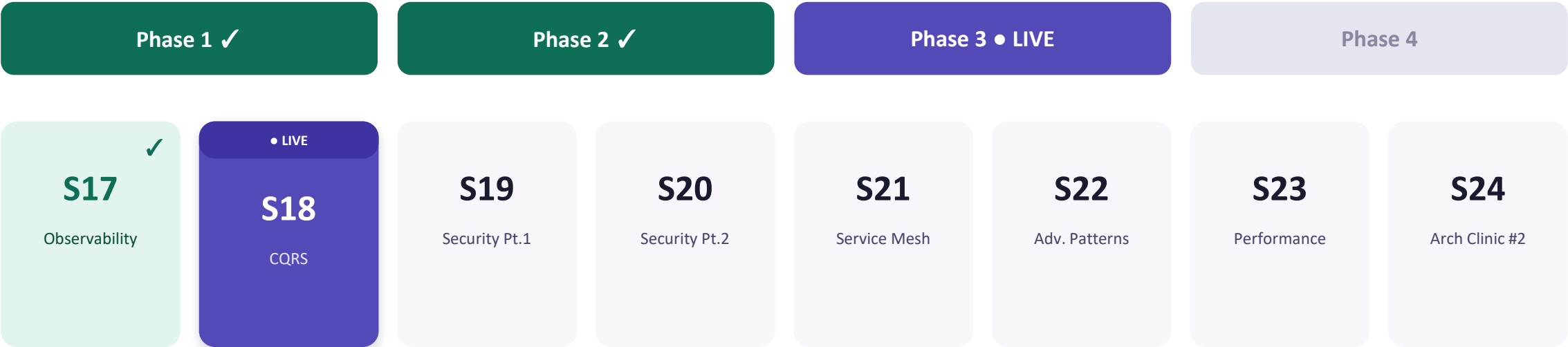


Phase 3 — Advanced & Enterprise



Session 18 of 29

Phase 3 — Advanced & Enterprise



Session 18 refactors product-service — the very first service you built in Session 1 — to separate its Command (write) and Query (read) responsibilities.

Note: the full 29-session roadmap lives in the Kickoff deck only.

Same Discipline as Session 12

"In Session 12 we built Saga Orchestration in plain Spring Boot — no Axon Framework — because the goal was to teach you the PATTERN so you could evaluate any framework independently afterward."

We implement CQRS with Spring Data — not Axon. Same philosophy as Session 12: learn the pattern, not the framework.

What's explicitly OUT of scope today

- Axon Framework — evaluate it independently once you understand the pattern it automates
- Event Sourcing — a separate, heavier pattern that often pairs with CQRS but is NOT required by it
- A physically separate read database — today's Read Model shares the same PostgreSQL database

THE PROBLEM

One Endpoint, Two Jobs

"product-service has grown since Session 1. It now has a Redis cache (Session 8), and Ops wants a new dashboard: 'show me every product, its category name, its current stock level from Inventory Service, and how many times it was ordered this week.'"

"A developer adds this logic straight into `ProductService.findAll()` — the same method the checkout page also calls for a simple product lookup."

Now checkout — a WRITE-adjacent, latency-sensitive path — is slowed down by a dashboard query nobody asked it to carry. One model. Two very different customers. Every change is now a compromise for someone.

🔗 Ask the class: "What would you do if a product manager asks for yet another dashboard field next month?" — it keeps piling into the same overloaded model unless something structural changes.

CQRS Command Query Responsibility Segregation — The Core Split



Command

Changes state

Changes state.
Represents intent ("create this product", "update this price").
Enforces business invariants.
Returns little or nothing — success/failure, an ID at most.



Query

Reads state

Reads state.
Never changes anything.
Shaped entirely around what the caller needs to DISPLAY or DECIDE with
— can be denormalized, joined, cached, or precomputed freely.

The core insight: once you stop forcing one model to serve both jobs, each side can evolve, be optimized, and be tested independently — at the cost of an explicit synchronization mechanism between them.

Write Model vs Read Model

Aspect	Write Model (Command)	Read Model (Query)
Purpose	Enforce invariants, persist correctly	Serve exactly what the consumer needs, fast
Shape	Normalized JPA entity (Product)	Flattened Projection/DTO (ProductSummaryProjection)
Changes	Rare — driven by domain rules	Frequent — driven by UI/reporting needs
Platform example	ProductCommandService.create/update/delete	ProductQueryService.findById/findAll

CQRS Scope — How Far Do You Go?

Simple CRUD, single model OK?



Skip CQRS

Read-heavy with complex queries?



CQRS — separate read model (today's build)

Write consistency + read performance both matter?



CQRS + Event

Audit trail needed?



CQRS + Event Sourcing (future pattern)

Team new to the codebase?



Start without CQRS

CQRS Is Not a Synonym for "Good Architecture"

"CQRS is not a synonym for 'good architecture.' It is a specific answer to a specific pain: read and write needs pulling a model in opposite directions. If you don't feel that pain yet, applying CQRS preemptively adds two models to keep mentally in sync and a synchronization mechanism to maintain — for no real benefit."

Today's refactor is justified: we DO have a real, described dashboard requirement pulling against a real, described write-side invariant set.

That is the bar to clear before reaching for this pattern in your own Capstone.

ProductCommandService Owns All Writes

The Command Service's ONLY job:

- `create()` / `update()` / `deleteById()` — the only methods that touch the database for writes
- Business invariants live HERE and ONLY here (e.g. `price > 0`)
- Publishes a `ProductChangedEvent` after every successful write — the read side reacts, the Command side does not know who is listening

? Naming discipline

None of the Command methods join categories, call Inventory, or compute aggregates — that temptation is now structurally harder to give in to.

ProductCommandService.java

```
@Service
public class ProductCommandService {
    @Autowired private ProductRepository productRepository;
    @Autowired private ApplicationEventPublisher eventPublisher;

    @Transactional
    public Product create(CreateProductRequest request) {
        Product product = new Product(null, request.getName(), request.getDescription(),
            request.getPrice(), request.getCategory());
        if (product.getPrice().compareTo(BigDecimal.ZERO) <= 0) {
            throw new InvalidProductException("Price must be positive"); // invariant lives HERE only
        }
        Product saved = productRepository.save(product);
        eventPublisher.publishEvent(new ProductChangedEvent(saved.getId(), "CREATED"));
        return saved;
    }
}
```

ProductCommandService.java

```
@Transactional
public Product update(Product product) {
    Product saved = productRepository.save(product);
    eventPublisher.publishEvent(new ProductChangedEvent(saved.getId(), "UPDATED"));
    return saved;
}

@Transactional
public void deleteById(Long id) {
    productRepository.deleteById(id);
    eventPublisher.publishEvent(new ProductChangedEvent(id, "DELETED"));
}
}
```

Invalid Price Rejected · Event Published

```
curl -X POST http://localhost:8081/api/v1/products \
  -d '{"name":"Bad","price":-1,"category":"ELECTRONICS"}'
# -> 400 Bad Request: InvalidProductException 'Price must be positive'

curl -X POST http://localhost:8081/api/v1/products \
  -d '{"name":"Laptop","price":999.99,"category":"ELECTRONICS"}'
# -> 201 Created
# [CQRS] ProductChangedEvent published: productId=42, changeType=CREATED
```

What this proves

Invariant validation and event publishing both live in exactly one place — the Command Service. The Query side never sees invalid data, and never has to know about validation rules at all.

Spring Data Projections — The Read Model

What is a Projection?

An interface (or DTO) that Spring Data can populate directly from a query, shaped for the reader's needs rather than mirroring the JPA entity. Spring Data generates the implementation — no manual mapping code, no second physical database.

Why this matters for our dashboard requirement:

- `ProductSummaryProjection` joins `Product` + `Category` — flattened, denormalized, exactly what the dashboard needs
- The Write Model (`Product` entity) never needs to know this shape exists
- `@Transactional(readOnly = true)` tells Hibernate to skip dirty-checking — a real performance gain on the read path

ProductSummaryProjection + Repository Query

```

public interface ProductSummaryProjection {
    Long getId();
    String getName();
    BigDecimal getPrice();
    String getCategoryName();          // flattened - write-side has categoryId only

    // Derived/computed field via a default method - read-side-only convenience
    default String getDisplayLabel() {
        return getName() + " (" + getCategoryName() + ")";    // read-only convenience
    }
}

// ProductRepository.java - add a projection-returning query alongside the existing entity methods
public interface ProductRepository extends JpaRepository<Product, Long> {
    @Query("SELECT p.id AS id, p.name AS name, p.price AS price, c.name AS categoryName " +
           "FROM Product p JOIN Category c ON p.categoryId = c.id WHERE p.id = :id")
    Optional<ProductSummaryProjection> findSummaryById(@Param("id") Long id);

    @Query("SELECT p.id AS id, p.name AS name, p.price AS price, c.name AS categoryName " +
           "FROM Product p JOIN Category c ON p.categoryId = c.id")
    List<ProductSummaryProjection> findAllSummaries();
}

```

ProductQueryService.java

```
@Service
public class ProductQueryService {
    @Autowired private ProductRepository productRepository;

    @Transactional(readOnly = true)    // hint to Hibernate: skip dirty-checking
    public Optional<ProductSummaryProjection> findById(Long id) {
        return productRepository.findSummaryById(id);
    }

    @Transactional(readOnly = true)
    public List<ProductSummaryProjection> findAll() {
        return productRepository.findAllSummaries();
    }
}
```

GET Returns categoryName — Flattened From the Join

```
curl http://localhost:8081/api/v1/products/42

{
  "id": 42,
  "name": "Laptop Pro",
  "price": 999.99,
  "categoryName": "ELECTRONICS"    // <- came from the JOIN, not stored on Product
}
```



No manual mapping code, no second database

Spring Data generated the implementation of ProductSummaryProjection entirely from the @Query annotation. The Query side and Write side now evolve independently.

Why an Event at All — If the DB Is Shared?

*Today's Read Model shares the SAME database as the Write Model — no synchronization lag exists at the data level.
So why publish an event?*

1

The Extension Seam

It is the seam where the pattern would extend to a real separate read store later (cache warm-up, search index, materialized view) — without touching ProductCommandService again.

2

Open/Closed Principle

It keeps the Command side ignorant of who cares about its writes. Today one listener exists (cache eviction). Tomorrow, a second listener could update a Grafana metric (Session 17!) without changing a single line of the Command service.

ProductChangedEvent + Cache Eviction Listener

```
// ProductChangedEvent.java
public record ProductChangedEvent(Long productId, String changeType) {}

// ProductCacheEvictionListener.java — connects CQRS back to Session 8's Redis cache
@Component
public class ProductCacheEvictionListener {
    @Autowired private CacheManager cacheManager;

    @EventListener
    public void onProductChanged(ProductChangedEvent event) {
        // Evict the Session 8 Redis cache entry so the next Query reflects the Command's result
        Objects.requireNonNull(cacheManager.getCache("products")).evict(event.productId());
        Objects.requireNonNull(cacheManager.getCache("products")).evict("all");
        log.info("[CQRS] Cache evicted for product {} due to {}", event.productId(), event.changeType());
    }
}

// NOTE: @EventListener here runs SYNCHRONOUSLY on the same thread as the Command by default.
// Add @Async (with @EnableAsync) only if the listener does slow work — see Common Issues table.
```

The Full Loop: Command → Event → Cache Evicted → Query

1

```
POST /api/v1/products/42 (update price)
```

2

```
ProductCommandService.update() saves + publishes ProductChangedEvent
```

3

```
ProductCacheEvictionListener evicts products::42 and products::all
```

4

```
GET /api/v1/products/42 → fresh price, served by ProductQueryService
```

[🔗](#) Demo live: call the Command, then immediately call the Query endpoint and show the Redis cache log line confirms eviction happened via the event — not because ProductCommandService called the cache directly.

Internal Events Today. Cross-Service Events? Already Built.

Today: internal Spring events (same JVM).

`ApplicationEventPublisher` + `@EventListener` stay entirely inside product-service — no network hop, no serialization, no broker.

Cross-service events? Already covered.

Kafka-based domain events between services were built in Session 7 (Choreography Saga) and Session 12 (Orchestration Saga) — this session does not repeat that work.

Command/Query Split on product-service

product-service · Builds on S1 entity/repository, S8 Redis caching · 25 min

1

ProductCommandService

Move all write logic + invariant checks here; publish ProductChangedEvent

2

ProductSummaryProjection + ProductQueryService

@Transactional(readOnly=true), returns the flattened Projection

3

Event-Driven Cache Eviction

Replace direct @CacheEvict with ProductCacheEvictionListener

4

Unit Tests

Minimum 2 tests: invalid price rejected, event published on create

What We Added Today

 Capability Added

Read/Write Separation on product-service. Dashboards and reporting queries can now evolve freely without risking the correctness or performance of the checkout-critical write path.

 Services Updated

product-service: ProductCommandService, ProductQueryService, ProductSummaryProjection, ProductChangedEvent + ProductCacheEvictionListener replacing direct @CacheEvict.

Platform Progress — 8-Segment Tracker





Daily Quiz

8 Questions · 10 Minutes · Google Forms or Kahoot

Command vs Query · Projections · readOnly=true · When NOT to use CQRS

5 Things That Will Go Wrong Today

Query endpoint still returns stale data after a Command

Evict BOTH the individual product key AND the 'all products' list key — same lesson as Session 8, Q4.

ApplicationEventListener never fires

Confirm `eventPublisher.publishEvent()` is actually called, and the Listener is a discovered `@Component`.

Projection query returns null for categoryName

Check the JPQL `@Query` join condition — a typo in the join column silently returns null.

Unit test fails with `NullPointerException` on `eventPublisher`

`@Mock ApplicationEventPublisher` must be included alongside `@Mock ProductRepository`.

`@Transactional(readOnly=true)` still shows dirty-checking overhead

Verify the annotation is on the `ProductQueryService` method itself, not only the repository interface.



Session 18 Complete

Read/Write Separation Active on product-service

Next: Session 19 — Security Part 1
Wednesday, 3:00–5:30 PM, Online